

Senior .NET Developer bo'lish uchun bilish kerak bo'lgan 100 ta Top Savol va Javoblar Guide

Ushbu qo'llanma **Senior .NET Developer** lavozimiga tayyorgarlik ko'rayotgan yoki o'z bilimlarini chuqurlashtirmoqchi bo'lgan dasturchilar uchun C#, CLR, ASP.NET Core, EF Core, Microservices, Arxitektura, Performance, Multithreading va Security bo'yicha eng muhim 100 ta savol va ularning chuqurlashtirilgan javoblarini o'z ichiga oladi.

Mundarija

- [C# Dasturlash Tili va Xotira Boshqaruvi (1-20)](#1-c-dasturlash-tili-va-xotira-boshqaruvi-1-20)
- [CLR Internals va JIT Kompilyatsiya (21-35)](#2-clr-internals-va-jit-kompilyatsiya-21-35)
- [ASP.NET Core va Web Arxitektura (36-55)](#3-aspnet-core-va-web-arxitektura-36-55)
- [EF Core va Ma'lumotlar Bazasi Optimallashtirish (56-70)](#4-ef-core-va-malumatlar-bazasi-optimallashtirish-56-70)
- [Software Arxitektura va Patternlar (71-85)](#5-software-arxitektura-va-patternlar-71-85)
- [Performance Tuning, Diagnostics va Xavfsizlik (86-100)](#6-performance-tuning-diagnostics-va-xavfsizlik-86-100)

1. C# Dasturlash Tili va Xotira Boshqaruvi (1-20)

1. **Stack** va **Heap** o'rtasidagi asosiy farqlar nimada va ular qanday ishlaydi?

Javob:

- **Stack:** LIFO (Last In, First Out) strukturali bo'lib, funktsiya chaqiruvlari, lokal o'zgaruvchilar va qiymat turlarini (Value Types) saqlaydi. Xotira ajratish va bo'shatish juda tez amalga oshadi (Stack Pointer siljiydi).
- **Heap:** Dinamik xotira bo'lib, havolali turlar (Reference Types - Object, Class, String va hk) saqlanadi. Xotirani tozalash Garbage Collector (GC) tomonidan amalga oshiriladi.

2. **Value Type** va **Reference Type** farqlari. **boxing** va **unboxing** nima va ularning performance'ga ta'siri qanday?

Javob:

- **Value Type** (**struct** , **enum** , **int** , **bool**): O'zgaruvchi qiymatning o'zini saqlaydi.
- **Reference Type** (**class** , **interface** , **delegate** , **string**): O'zgaruvchi Heap'dagi obyekt manziliga (reference) ishora qiladi.
- **Boxing:** Value type'ni Reference type'ga (masalan, **object** ga) o'girish. Heap'da obyekt yaratiladi va qiymat nusxalanadi (sekin va GC pushaymonligi).
- **Unboxing:** Heap'dagi boxing qilingan qiymatni qayta Value type'ga ajratib olish.

3. **struct** va **class** o'rtasidagi farqlar nimalardan iborat va qachon **struct** ishlatish kerak?

Javob:

- `struct` — Value type (Stack'da saqlanadi, vorislikni qo'llamaydi, sukut bo'yicha kichik hajmli immutable ma'lumotlar uchun mos).
- `class` — Reference type (Heap'da saqlanadi, OOP imkoniyatlarini to'liq qo'llaydi).
- **Qachon `struct` ishlatish kerak?** Hajmi 16 baytdan kichik bo'lsa, tez-tez yaratilib yo'qotilsa, immutable bo'lsa va unboxing/boxing xavfi bo'lmasa.

4. `readonly struct` va `ref struct` nima va ular xotira optimallashtirishda qanday rol o'ynaydi?

Javob:

- **`readonly struct`**: Struct'ning barcha maydonlari `readonly` ekanligini kafolatlaydi va C# kompilyatori defensive copying (himoya nusxalash) ishlatishining oldini oladi.
- **`ref struct` (`Span` kabi)**: Obyekt faqat Stack'da joylashishini majburiy qiladi. U Heap'ga (boxing, class maydoni, async metodlar kabi joylarga) o'tib ketmaydi.

5. `Span` va `Memory` nima? Ularning farqi va ishlatilish o'rinlari.

Javob:

- **`Span`**: Stack-only `ref struct` bo'lib, uzluksiz xotira (Array, Stackalloc, Unmanaged memory, String) bo'laklarini GC allocation'siz ko'rish va tahrirlash imkonini beradi. Async metodlarda ishlatib bo'lmaydi.
- **`Memory`**: `Span` ga o'xshash, lekin `struct` bo'lgani uchun Heap'da ham saqlanishi va `async/await` metodlarda ishlatilishi mumkin. `Span` ga `.Span` xossasi orqali o'giriladi.

6. `ref`, `out` va `in` kalit so'zlarining farqi nimada?

Javob:

- `ref`: Metodga o'zgaruvchi havolasini beradi. Metodgacha boshlang'ich qiymat berilgan bo'lishi shart.
- `out`: Metodga havola beriladi, lekin metod ichida unga albatta qiymat tayinlanishi shart.
- `in`: Metodga havolani `readonly` rejimida beradi (nusxalamaslik va o'zgartirmaslik uchun).

7. Garbage Collector (GC) qanday ishlaydi va Generation (Gen 0, Gen 1, Gen 2) tushunchalari nima?

Javob:

- **Gen 0**: Yangi yaratilgan qisqa o'mirli obyektlar (Lokal o'zgaruvchilar). GC eng tez tozalaydigan qism.
- **Gen 1**: Gen 0 tozalanganda tirik qolgan obyektlar (Bufer/o'tish zonasi).
- **Gen 2**: Uzoq yashaydigan obyektlar (Singleton servislar, static o'zgaruvchilar, keshlar).
- **LOH (Large Object Heap)**: 85,000 baytdan katta obyektlar (masalan, katta arraylar) to'g'ridan-to'g'ri LOH ga tushadi va Gen 2 bilan birga tozalanadi.
- **POH (Pinned Object Heap)**: .NET 5+ da joyi o'zgarmasligi kerak bo'lgan obyektlar uchun.

8. `IDisposable` va `Finalizer` (~Destructor) o'rtasidagi farq nimada? `Dispose Pattern` qanday yoziladi?

Javob:

- **`IDisposable`**: Unmanaged resurslarni (fayl, DB connection, socket) **deterministik** (dasturchi tomonidan belgilangan vaqtda) bo'shatish uchun.

- **Finalizer** : GC obyektini xotiradan o'chirayotganda unmanaged resurslarni bo'shatish uchun **ed-hoc** chaqiriladigan metod.
- **Dispose Pattern**:

```
public class ResourceHolder : IDisposable
{
    private bool _disposed;
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
    protected virtual void Dispose(bool disposing)
    {
        if (!_disposed)
        {
            if (disposing) { / managed resurslar / }
            / unmanaged resurslar /
            _disposed = true;
        }
    }
    ~ResourceHolder() => Dispose(false);
}
```

9. **GC.SuppressFinalize** nima uchun chaqiriladi?

Javob:

GC obyektida Finalizer bor bo'lsa, uni birdan o'chira olmaydi, uni Finalization Queue'ga joylashtiradi (obyekt Gen 2 ga suriladi). `GC.SuppressFinalize(this)` chaqirilganda, GC'ga "bu obyekt resurslarini dasturchi `Dispose` orqali tozaladi, Finalizer'ni chaqirish shart emas" deb aytiladi va obyekt birinchi urushdayoq toza o'chiriladi.

10. **Task** va **ValueTask** o'rtasidagi farq nima va qachon **ValueTask** ishlatish tavsiya etiladi?

Javob:

- **Task** — Reference type. Har bir async operatsiya uchun Heap'da obyekt yaratadi.
- **ValueTask** — Struct type. Agar async metod ko'pincha synchronous (masalan, keshdan) natija qaytarsa, Heap allocation bo'lmaydi.
- **Qachon ValueTask** : Metod natijasi ko'p hollarda synchronous bo'lganda (masalan `if (_cache.TryGetValue(...)) return new ValueTask(data)`).

11. **async/await** qanday ishlaydi va kompyator uni qanday kodga o'giradi (State Machine)?

Javob:

C# kompyatori `async` metodni `IAsyncStateMachine` interfeysini amalga oshiruvchi `struct` yoki `class` ga aylantiradi. Metod ichida `await` uchraganda, resurs kutilayotgan operatsiyaga ulanadi (continuation) va thread ThreadPool'ga qaytariladi. Operatsiya tugagach, ThreadPool thread'laridan biri kadrni davom ettiradi.

12. `ConfigureAwait(false)` nima uchun kerak va .NET Core / ASP.NET Core da u shartmi?**Javob:**

`ConfigureAwait(false)` operatsiyadan so'ng kutilayotgan kodni asl `SynchronizationContext` (masalan, UI thread) da davom ettirmaslikni aytadi (deadlock'ning oldini oladi).

ASP.NET Core da custom `SynchronizationContext` yo'q, shuning uchun kutubxonalardan tashqari (Library code) ASP.NET Core controller/service darajasida `ConfigureAwait(false)` shart emas.

13. `Thread`, `ThreadPool`, `Task` va `Parallel` o' me'zonlari farqi.**Javob:**

- `Thread` : Operatsion tizimning quyi darajali ipi (qimmat resurs).
- `ThreadPool` : Tayyor thread'lar hovuzi, thread yaratish xarajatlarini kamaytiradi.
- `Task` : Asinxron operatsiyaning yuqori darajali abstraksiyasi (`ThreadPool`'da ishlaydi).
- `Parallel` : CPU-intensive ishlarni bir nechta thread'ga bo'lib parallel bajarish uchun (Fork-Join).

14. `Lock`, `Monitor`, `Mutex`, `SemaphoreSlim` va `Interlocked` farqlari va ishlatilish holatlari.**Javob:**

- `lock` / `Monitor` : Bitta jarayon (process) ichidagi thread-safety uchun (`lock` `Monitor` ning sintaktik shakaridir).
- `Mutex` : Tizim (OS) darajasida kross-protsess blokirovka uchun.
- `SemaphoreSlim` : Bir vaqtning o'zida N ta thread'ga kirishga ruxsat berish (async/await qo'llaydi: `WaitAsync`).
- `Interlocked` : Atomar amallar (Increment, Add, Exchange) uchun eng tez va lock-free yechim.

15. `Channel` nima va u producer-consumer ssenariylarida `BlockingCollection` dan nimasi bilan afzal?**Javob:**

`System.Threading.Channels` asinxron, yuqori unumdorlikka ega Producer-Consumer navbati hisoblanadi. U `BlockingCollection` dan ko'ra asinxron kutilmalarni to'liq qo'llaydi va sezilarli darajada kamroq allocation talab qiladi.

16. `Record` turlari nima (`record class`, `record struct`) va `with` expression qanday ishlaydi?**Javob:**

- `Record` : Immutability va Value-based equality (qiymat bo'yicha tenglik) xususiyatlariga ega bo'lgan tur.
- `with` : Mavjud record nusxasi asosida faqat ba'zi xossalari o'zgartirilgan yangi immutable nusxa yaratish imkonini beradi (Nondestructive mutation).

17. `Delegate` va `Event` nimasi bilan farq qiladi? `Func`, `Action`, `Predicate` nima?**Javob:**

- `Delegate` : Metod havolasini (pointer) saqlaydigan tur xavfsiz obyekt.

- **Event** : Delegate ustidagi o'rab turuvchi abstraksiya bo'lib, tashqaridan faqat **+=** (subscribe) va **-=** (unsubscribe) qilishga ruxsat beradi, tashqaridan birdaniga chaqirib yoki nolga tenglab bo me bo'lmaydi.
- **Action** : Natija qaytarmaydigan delegate (**void**).
- **Func** : Natija qaytaradigan delegate.
- **Predicate** : **bool** qaytaradigan delegate (**Func**).

18. Pattern Matching imkoniyatlari (Switch expressions, Property patterns, Positional, Relational, List patterns).

Javob:

C# 8+ dan boshlab ma'lumotlar tuzilishi va tiplari bo'yicha shartlarni ixcham yozish imkonini beradi:

```
string category = item switch {
    Product { Price: > 100, Stock: > 0 } => "Premium",
    [1, 2, ..] => "Sequence",
    _ => "Standard"
};
```

19. Source Generators nima va ular Reflection o'rnini qanday egallaydi?

Javob:

Source Generator — kompilyatsiya vaqtida (Compile-time) kodni tahlil qilib, yangi C# fayllarini dinamik generatsiya qiladigan rosmaniy vosita. U Reflection'ning runtime xarajatlarini va JIT overhead'ini nolga tushiradi (masalan, System.Text.Json generatorlari).

20. **Unsafe** code va **fixed** statement nima uchun ishlatiladi?

Javob:

- **Unsafe** : Pointerlar bilan to'g'ri-to'g me ishlash, unmanaged xotiraga murojaat qilish imkonini beradi.
- **fixed** : Pointer ishlatilayotgan paytda GC managed obyekttni Heap'da surib yubormasligi uchun o'sha obyekttni xotirada qotirib (pin qilib) turadi.

2. CLR Internals va JIT Kompilyatsiya (21-35)

21. C# kodi qanday ijro etiladi? (Source Code -> C# Compiler -> CIL/IL -> JIT -> Native Code).

Javob:

C# kodi **roslyn** kompyatori orqali birinchi bo'lib **CIL (Common Intermediate Language)** va **Metadata** ga o'tkaziladi va **.dll / .exe** faylga joylanadi. Dastur ishga tushganda CLR (Common Language Runtime) ushbu IL kodni **JIT (Just-In-Time)** kompyatori orqali protsessorning **Native Machine Code** siga aylantiradi.

22. JIT Kompilyatsiya turlari: Normal JIT, Tiered Compilation va Profile-Guided Optimization (PGO) nima?

Javob:

- **Tiered Compilation:**
- *Tier 0:* Dastur tez ishga tushishi uchun kod minimallashtirilgan optimallashtirish bilan tezda JIT qilinadi.
- *Tier 1:* Ko'p chaqiriladigan metodlar ("hot code") fonda chuqur optimallashtirish bilan qayta JIT qilinadi.
- **PGO (Dynamic PGO):** .NET 8 da Tier 0 bajarilish jarayonida to'plangan statistik ma'lumotlar (qaysi tiplar ko'p keldi, qaysi tarmoqlar ko'p bajarildi) asosida Tier 1 da maksimal darajada mukammal Native kod yaratadi.

23. Native AOT (Ahead-Of-Time) nima va uning JIT dan afzalliklari va cheklovlari nimalardan iborat?

Javob:

- **Afzalliklari:** JIT va CLR infratuzilmasiz to'g'ridan-to'g'ri Native ikkilik fayl (binary) yaratadi. Ishga tushish vaqti (Startup time) juda tez va xotira (Memory footprint) juda kam bo'ladi.
- **Cheklovlari:** Dinamik Reflection, `Assembly.Load`, dinamik kod generatsiyasi cheklangan.

24. AssemblyLoadContext (ALC) nima va u pluginlar (Plugin architecture) arxitekturasida qanday ishlatiladi?

Javob:

ALC — bu assembly'larni xotiraga yuklash va ajratish isolatsiyasi. U orqali bitta ilovaning o'zida bir xil kutubxonaning har xil versiyalarini yuklash va plugin ishini tugatgach, u yuklagan assembly'larni xotiradan (Unload) to'liq o'chirib tashlash mumkin.

25. Obyektning Heap'dagi xotira tuzilishi (Object Header, TypeHandle, Fields).

Javob:

Heap'dagi har bir obyekt quyidagi qismlardan iborat:

- **Object Header (8 bayt x64 da):** SyncBlockIndex (lock, hashcode va GC belgilari uchun).
- **TypeHandle / MethodTable Pointer (8 bayt):** Obyekt tipining CLR dagi metadata jadvaliga ishora.
- **Fields:** Obyektning xususiyat va maydon ma'lumotlari.

26. Virtual Method Table (vtable) qanday ishlatiladi?

Javob:

Polimorfizm bilan ishlaganda obyektning MethodTable jadvalida `vtable` (virtual metodlar manzillari ro'yxati) bo'ladi. Chaqiruv vaqtida CLR obyektning haqiqiy tipiga qarab `vtable` dagi to'g'ri metod manzilini topadi va chaqiradi (Virtual dispatch overhead).

27. Exception Handling ichki mexanizmi: First-chance va Second-chance exception nima?

Javob:

- **First-chance:** Xatolik yuzaga kelishi bilanoq CLR debugger va handler'larga xabar beradi.
- **Second-chance:** Agar hech qaysi `try-catch` blok xatolikni ushlamasa, dastur qulashidan oldin so'nggi imkoniyat beriladi va javobsiz qolsa Process tugatiladi.

- *Eslatma*: Exception yaratish Stack Walk qilgani uchun performance uchun juda qimmatga tushadi.

28. String immutability sababi va String Interning nima?

Javob:

- **Immutability**: String o'zgarmasdir. Bu thread-safety va keshlanishni ta'minlaydi.
- **String Interning**: CLR bir xil string literal'larni xotirada faqat bitta nusxada saqlaydi (`String.Intern`). Bu xotirani tejaydi.

29. Equality (Tenglik) turlari: ReferenceEquals , Equals , == operatori va GetHashCode() shartlari.

Javob:

- `ReferenceEquals` : Ikkala o'zgaruvchi xotirada bir xil manzilga ko'rsatayotganini tekshiradi.
- `Equals` / `==` : Obyekt ma'nosi bo'yicha tengligini tekshiradi (overriding qilinishi mumkin).
- **Altin Qoida**: Agar `Equals` ni override qilsangiz, albatta `GetHashCode()` ni ham mos ravishda override qilishingiz shart, aks holda `Dictionary` va `HashSet` da xatoliklar yuzaga keladi.

30. Expression Trees nima va LINQ to SQL u orqali C# kodini SQL ga qanday aylantiradi?

Javob:

Expression Tree — koddagi amallarni ma'lumotlar darajasi (Tree struktura) sifatida ifodalaydi (`Expression>`). EF Core ushbu daraxtni tahlil qilib (parse), C# shartlarini tegishli ma'lumotlar bazasining SQL dialect'iga tarjima qiladi.

31. Direct call, Virtual call va Interface call o'rtasidagi performance farqlari.

Javob:

- **Direct call**: Kompilyatsiya vaqtida manzil aniq, eng tez chaqiruv (Inlining bo'lishi mumkin).
- **Virtual call**: `vtable` orqali manzil topiladi (bitta qo'shimcha indirection).
- **Interface call (Devirtualization/Interface dispatch)**: Interface orqali chaqirish chaqirilayotgan turga qarab o'zgargani uchun bir oz murakkabroq vtable (ITable) qidirishni talab qiladi. .NET Core Devirtualization orqali buni ko'p hollarda Direct call ga aylantiradi.

32. Method Inlining nima va [MethodImpl(MethodImplOptions.AggressiveInlining)] qachon ishlatiladi?

Javob:

Inlining — JIT kichik metod chaqiruvini o'sha chaqirilgan joyga to'g'ridan-to'g'ri kod sifatida joylashtirib, metod chaqirish xarajatlarini (stack frame yaratish) yo'qotadi. `AggressiveInlining` ko'p chaqiriladigan juda kichik metodlarga qo'yiladi.

33. Out-Of-Memory (OOM) Exception faqat RAM to'lganda chiadimi? Fragmentatsiya nima?

Javob:

Yo'q, OOM umumiy RAM to'lmagan bo'lsa ham chiqishi mumkin. Masalan, Heap fragmentatsiyasi tufayli uzluksiz ketma-ketlikdagi blok (masalan katta array uchun) topilmasa yoki 32-bitli process manzil fazosi tugab qolsa OOM beradi.

34. SIMD (Single Instruction Multiple Data) va Hardware Intrinsics nima?

Javob:

SIMD — protsessorning maxsus vektor registrilaridan (AVX, SSE) foydalanib, bitta sikl operatsiyasida bir nechta ma'lumotlar (masalan, 8 ta int soni) ustida parallel amal bajarish imkoniyatidir.

35. CoreCLR da GC Audit va Diagnostic Server (EventPipe, IPC socket) qanday ishlaydi?

Javob:

CoreCLR maxsus Unix Domain Socket yoki Named Pipe orqali diagnostika xabarlarini uzatadi. `dotnet-trace`, `dotnet-dump` kabi uskunalar EventPipe orqali ilovani to'xtatmasdan telemetriya va profayling ma'lumotlarini yig'ib oladi.

3. ASP.NET Core va Web Arxitektura (36-55)

36. ASP.NET Core Request Pipeline va Middleware qanday ishlaydi?

Javob:

ASP.NET Core so'rovlar zanjiri (pipeline) **Russian Doll (Matryoshka)** modeliga asoslangan. Har bir Middleware so'rovni qabul qiladi, o'z amallarini bajaradi va `next()` orqali keyingi middleware'ga uzatadi. Qaytishda esa javob zanjir bo'ylab teskari yo'nalishda qaytadi.

37. Middleware va Filter (Action, Exception, Resource, Result) o'rtasidagi farq nimada?

Javob:

- **Middleware:** Butun HTTP darajasida ishlaydi, Routing sodir bo'lishidan oldin ham so'rovni ushlab qolishi mumkin. MVC contextual ma'lumotlaridan bexabar.
- **Filters:** ASP.NET Core MVC / API karkasi ichida, ya'ni Routing va Action tanlangandan so'ng ishlaydi va Controller/Action kontekstiga ega bo'ladi.

38. Dependency Injection Lifetime: Transient, Scoped, Singleton va Captive Dependency muammosi.

Javob:

- **Transient** : Har safar so'ralganda yangi instance yaratiladi.
- **Scoped** : Bitta HTTP Request davomida bitta instance yaratiladi.
- **Singleton** : Ilova umri davomida bitta instance yaratiladi.
- **Captive Dependency**: Katta umrga ega servis (masalan, Singleton) kichikroq umrga ega servisni (masalan, Scoped) konstruktorida ushlab qolsa sodir bo'ladi. Bu ma'lumotlar sizishiga va xatoliklarga olib keladi.

39. Kestrel Server arxitekturasi va u Reverse Proxy (Nginx / IIS / YARP) orqasida qanday ishlaydi?

Javob:

Kestrel — ASP.NET Core uchun juda tez, kross-platformali, asinxron I/O va Socket portlariga asoslangan veb-server.

Odatda Nginx/YARP reverse proxy orqasida ishlatiladi. Nginx TLS termination, Rate limiting, Static file serving va xavfsizlik qatlamini bajarsa, Kestrel faqat ilova logikasini bajaradi.

40. Options Pattern: `IOptions`, `IOptionsSnapshot` va `IOptionsMonitor` farqlari.

Javob:

- `IOptions`: Singleton bo'lib, konfiguratsiyani ilova ishga tushganda bir marta o'qiydi (o'zgarishlarni sezmaydi).
- `IOptionsSnapshot`: Scoped bo'lib, har bir HTTP request da konfiguratsiyaning yangi qiymatini o'qiydi.
- `IOptionsMonitor`: Singleton bo'lib, fayl diskda o'zgarganda real vaqtda hodisa (OnChange) beradi va yangi qiymatni taqdim etadi.

41. Authentication va Authorization o me'zonlari va Pipeline'dagi o'rni.

Javob:

- **Authentication (Kimligini tasdiqlash):** Foydalanuvchi shaxsini aniqlash (`ClaimsPrincipal` yaratish).
- **Authorization (Ruxsat berish):** Foydalanuvchining ushbu resursga yetarlicha huquqi borligini tekshirish (Role, Claim, Policy asosida).
- Pipeline'da birinchi Authentication middleware, so'ngra Authorization middleware joylashishi shart.

42. Custom Policy-Based Authorization qanday yaratiladi? (`IAuthorizationRequirement` va `AuthorizationHandler`).

Javob:

Dasturda maxsus talab (Requirement) yaratiladi va u bo'yicha `AuthorizationHandler` yoziladi. Handler ichida biznes shartlar (masalan: foydalanuvchi yoshi > 18 yoki muayyan Hujjat egasi bo'lishi) tekshirilib `context.Succeed(requirement)` chaqiriladi.

43. JWT Token arxitekturasi: Token formati, Refresh Token pattern va Security Key aylantirish (Rotation).

Javob:

JWT 3 qismdan iborat: `Header.Payload.Signature`.

- Security Key orqali Server payload o'zgartirilmaganini tekshiradi.
- **Refresh Token Pattern:** Qisqa muddatli Access Token va uzoq muddatli Refresh Token (DB da saqlanadigan va revoke qilinishi mumkin bo'lgan) orqali seans xavfsiz boshqariladi.

44. `BackgroundService` va `IHostedService` qanday ishlaydi? Ularda `Scoped` servislar qanday ishlatiladi?

Javob:

`BackgroundService` — fonda uzluksiz ishlaydigan vazifa (`ExecuteAsync`).

U Singleton bo'lgani uchun u ichida Scoped servislardan foydalanish uchun konstruktordan

`IServiceScopeFactory` olib, `using (var scope = _scopeFactory.CreateScope())` orqali alohida scope yaratish kerak.

45. Rate Limiting ASP.NET Core 7+ da qanday sozlanadi? (Fixed Window, Sliding Window, Token Bucket, Concurrency).

Javob:

ASP.NET Core 7+ o'zining bevosita Rate Limiting middleware'iga ega.

- **Fixed Window:** Vaqt bo'lagida qat'iy limit.
- **Sliding Window:** Vaqt bo'lagi siljish dinamikasiga ega.
- **Token Bucket:** Tokenlar to'planib boradi va so'rov kelganda sarflanadi.
- **Concurrency Limit:** Bir vaqtning o'zida parallel bajarilishi mumkin bo'lgan maksimal so'rovlar soni.

46. ASP.NET Core gRPC servislari nima va u REST API dan nimasi bilan ustun?

Javob:

gRPC — HTTP/2 va Protocol Buffers (Protobuf) ga asoslangan yuqori unumdorlikka ega RPC karkasi. U ikkilik (binary) serialization ishlatadi, kamroq tarmoq trafigi sarflaydi, bi-directional streaming qo'me'llaydi va kontratga (.proto) qat'iy tayanadi.

47. WebSockets va SignalR o'me me'zonlari va Hub arxitekturasini.

Javob:

SignalR — real vaqtda ikki tomonlama aloqa o'rnatish uchun mo'ljallangan abstraksiya. U eng yaxshi transport turini avtomatik tanlaydi (WebSockets -> Server-Sent Events -> Long Polling). Hub klassi klientlar bilan metodlar orqali muloqot qiladi.

48. Minimal APIs va Controller-based APIs o'rtasidagi farqlar va ularning unumdorligi.

Javob:

Minimal API yengil, minimal overhead'ga ega va ilovani tez ishga tushirish uchun mos. U MVC Controller'lariga qaraganda kamroq qatlamlarga ega va Routing/Execution tezroq bajariladi.

49. CORS (Cross-Origin Resource Sharing) qanday ishlaydi va Preflight (OPTIONS) so'rovi nima?

Javob:

CORS — brauzerning bir domendan boshqa domenga noaniq so'me me'rovlar yuborishini cheklovchi xavfsizlik mexanizmi.

Murakkab so'rovlarda (Custom Headers, PUT, DELETE) brauzer birinchi OPTIONS (Preflight) so'me'rovini yuborib, server ruxsatini olgach, haqiqiy so'rovni uzatadi.

50. Health Checks ASP.NET Core da qanday sozlanadi (Liveness, Readiness, Startup probes)?

Javob:

Kubernetes va monitoring vositalari uchun AddHealthChecks() orqali DB, Redis va tashqi API holati tekshiriladi.

- **Liveness:** Ilova tirikmi? (Ishlamasa container restart qilinadi).
- **Readiness:** Ilova so'me'rovlarni qabul qilishga tayyormi? (DB bilan aloqa bormi?).

51. Output Caching vs Response Caching ASP.NET Core da.

Javob:

- **Response Caching:** HTTP header'lariga (`Cache-Control`) tayanadi, asosan brauzer va intermediate proxy'lar uchun.
- **Output Caching (ASP.NET Core 8+):** Server tomonda to'liq sozlanadigan, keshni teglash (tagging) va keshni dinamik bekor qilish (invalidation) imkonini beruvchi server keshirish tizimi.

52. Content Negotiation nima va custom `OutputFormatter` qanday yoziladi?

Javob:

Content Negotiation — klient yuborgan `Accept` header'iga qarab (masalan `application/json` yoki `application/xml`) server qaysi formatda javob qaytarishini aniqlashi. Custom formatter uchun `TextOutputFormatter` dan inheritance olinadi.

53. Exception Handling Best Practices: `ProblemDetails` (RFC 7807) va `ExceptionHandler` (ASP.NET Core 8+).

Javob:

ASP.NET Core 8 da `ExceptionHandler` interfeysi orqali markazlashgan xatoliklarni ushlab va standart RFC 7807 formatida `ProblemDetails` javobini shakllantirish tavsiya etiladi.

54. Idempotency nima va API da Idempotent so'rovlar qanday ta'minlanadi?

Javob:

Idempotency — so me'rov necha marta takrorlanishidan qat'i nazar, tizim holati bir marta o'zgargandek qolishi. REST da GET, PUT, DELETE idempotent. POST so me'rovi uchun `Idempotency-Key` header'i va Redis'da kesh shablonidan foydalaniladi.

55. OpenTelemetry va Structured Logging (Serilog/ILogger) integration.

Javob:

Structured Logging ma'lumotlarni text o'rniga JSON obyekti sifatida (Properties: UserId, OrderId) saqlaydi. OpenTelemetry bilan birgalikda Distributed Tracing (TraceId, SpanId) hosil qilib, microservice'lar bo'ylab so me'rov izini ko'rish imkonini beradi.

4. EF Core va Ma'lumotlar Bazasi Optimallashtirish (56-70)

56. EF Core Change Tracker qanday ishlaydi? `State` turlari.

Javob:

Change Tracker `DbContext` xotirasidagi barcha obyektlarni kuzatadi. Obyekt holatlari (EntityState):

- `Detached` (kuzatilmayapti)
- `Unchanged` (o'zgarmagan)
- `Added` (yangi qo'shilgan)

- `Modified` (o'zgargan)
- `Deleted` (o me'chirilishi kerak)

57. `AsNoTracking` va `AsNoTrackingWithIdentityResolution` qachon ishlatiladi?

Javob:

- `AsNoTracking` : Faqat o'qish (Read-only) so me me'rovlari uchun. Change Tracker obyektlarni saqlamaydi, natijada xotira va vaqt tejiladi.
- `AsNoTrackingWithIdentityResolution` : Bir so me me'rovda bitta obyekt takroran kelsa (masalan Graph so me me'rovlarda), xotirada bitta instance bo'lishini ta'minlaydi.

58. EF Core N+1 query muammosi nima va u qanday hal qilinadi?

Javob:

N+1 muammosi — asosiy obyektidan 1 ta so me'rov olinib, unga bog'liq har bir N ta sub-obyekt uchun alohida so me'rov yuborilishi.

Yechimi: Eager Loading (`Include` , `ThenInclude`), Explicit Loading yoki Proyeksiya (`Select`) ishlatish.

59. `Include` vs `Select` (Projection) performance farqi.

Javob:

`Include` entity'ning barcha ustunlarini yuklaydi. `Select` (Projection) esa faqat kerakli ustunlarni SQL darajasida tanlab oladi va `DTO` ga o'giradi. Projection so me'rovlarni maksimal optimallashtiradi va ortiqcha ma'lumot yuklanishining oldini oladi.

60. Split Queries (`AsSplitQuery`) nima uchun kerak va Cartesian Explosion nima?

Javob:

Bir nechta `Include` bo me'lsa, SQL `JOIN` lar tufayli jadval qatorlari ko'payib ketadi (**Cartesian Explosion**). `AsSplitQuery` so me'rovni bir nechta alohida oddiy SQL so me'rovlariga bo'lib bajaradi.

61. DbContext Pooling (`AddDbContextPool`) qanday ishlaydi va nimasi bilan afzal?

Javob:

Har bir HTTP request uchun yangi `DbContext` instance yaratish o'rniga pool'dagi mavjud instance qayta ishlatiladi. Bu xotira ajratish va GC yuklamasini keskin kamaytiradi.

62. Global Query Filters va Soft Delete pattern.

Javob:

`OnModelCreating` da `builder.Entity().HasQueryFilter(e => !e.IsDeleted)` orqali barcha SQL so me'rovlariga avtomatik shart qo'shiladi. So me'rovlardan buni olib tashlash uchun `IgnoreQueryFilters()` ishlatiladi.

63. Compiled Queries (`EF.CompileAsyncQuery`) qachon ishlatiladi?

Javob:

Juda tez-tez bajariladigan standart so'rovlar uchun EF Core har safar LINQ Expression Tree'ni tahlil qilmasligi uchun SQL so'rov shablonini keshlab qo'yadi.

64. EF Core da Transactions: Explicit Transaction, `ExecutionStrategy` va Retry Logic.

Javob:

Aralash amallar uchun `Database.BeginTransactionAsync()` ishlatiladi. Masalan SQL Server transient xatolarida Avto-retry sozlanganda (`EnableRetryOnFailure`), transaktsiyalarni `ExecutionStrategy.ExecuteAsync` ichida yozish shart.

65. Optimistic Concurrency Control vs Pessimistic Locking.

Javob:

- **Optimistic Concurrency:** Ma'lumotlarni bloklamaydi, versiya ustuni (`[Timestamp]` / `RowVersion`) ishlatadi. O'zgartirish vaqtida versiya mos kelmasa `DbUpdateConcurrencyException` tashlaydi.
- **Pessimistic Locking:** Bazadagi qatorni o'qish vaqtida SQL `FOR UPDATE` orqali boshqa klientlardan qat'iy bloklab turadi.

66. Shadow Properties va Property Bag Entities.

Javob:

- **Shadow Properties:** C# entity klassida yo'q, lekin EF Core modelida va ma'lumotlar bazasi jadvalida bor bo'lgan ustunlar (masalan, `LastModifiedDate`).
- **Property Bag:** Entity alohida class emas, `Dictionary` sifatida saqlanishi.

67. MSSQL da Index turlari: Clustered, Non-Clustered, Included Columns va Composite Indexing.

Javob:

- **Clustered Index:** Jadval ma'lumotlarining haqiqiy fizik joylashuvi (bitta jadvalda faqat 1 ta bo'ladi, odatda PK).
- **Non-Clustered Index:** B-Tree strukturasida ma'lumot ko'rsatgichlarini saqlaydi.
- **Included Columns:** Ko'p so'raladigan ustunlarni index barglariga qo'shib qo'yish (Key Lookup bo'lmasligi uchun).

68. Key Lookup (Bookmark Lookup) va Index Scan vs Index Seek farqlari.

Javob:

- **Index Seek:** Index B-Tree bo'ylab ma'lumotni darhol topish (Eng tez).
- **Index Scan:** Butun index barglarini ketma-ket o'qib chiqish.
- **Key Lookup:** Non-clustered index'da yo'q bo'lgan ustunni Clustered index'dan borib qo'shimcha o'qib kelish (sekin).

69. Deadlock nima va MSSQL da Deadlock'larning oldi qanday olinadi?

Javob:

Deadlock — ikkita transaktsiya bir-birining bloklagan resursini kutilganda sodir bo'ladi.

Oldini olish: Transaktsiyalarda jadvallarga murojaat qilish tartibini bir xil qilish, transaktsiyalarni qisqa qilish va mos index'lar qo'shish.

70. Outbox Pattern va MassTransit / Rebus bilan Transactional Messaging.

Javob:

Ma'lumotlar bazasiga yozish va Message Broker'ga (RabbitMQ) event yuborish bir vaqtda sodir bo'lishi uchun, event avval DB'dagi `OutboxMessages` jadvaliga yoziladi (bir transaktsiyada). Fondagi worker esa ularni brokerga kafolatli yuboradi (At-least-once delivery).

5. Software Arxitektura va Patternlar (71-85)

71. Clean Architecture va Hexagonal (Ports & Adapters) arxitekturasi tamoyillari.

Javob:

Markazda biznes logika va Domain modellari yotadi. Tashqi bog'liqliklar (DB, UI, External API) ichki qatlamga bog'liq bo'ladi, lekin ichki qatlam tashqi dunyoni bilmaydi (Dependency Inversion Principle).

72. Domain-Driven Design (DDD): Aggregate, Root, Entity, Value Object va Domain Events.

Javob:

- **Entity:** O'z identifikatoriga (Id) ega obyekt.
- **Value Object:** Id'si yo'q, faqat qiymat bo'yicha teng bo'ladigan immutable obyekt (masalan, `Money`).
- **Aggregate Root:** Bir guruh entity va value object'larni yagona tranzaktsion chegaraga oladigan asosiy kirish nuqtasi.
- **Domain Event:** Biznesda yuz bergan muhim voqea (masalan, `OrderCreatedEvent`).

73. CQRS (Command Query Responsibility Segregation) pattern va MediatR library.

Javob:

CQRS — tizimni Ma'lumot yozish (Commands) va Ma'lumot o'qish (Queries) modellariga ajratish. MediatR kutubxonasi in-memory bus vazifasini bajarib, Controller va Handler'larni bo'sh bog'lash (decoupling) uchun ishlatiladi.

74. Event Sourcing va CQRS birgalikda qanday ishlaydi?

Javob:

Event Sourcing'da obyekt joriy holati saqlanmaydi, balki unga yuz bergan barcha hodisalar (events) ketma-ketligi saqlanadi. CQRS o'qish qatlamini ushbu event'lardan **Read Model** (Projections) hosil qilish uchun ishlatadi.

75. Microservices Communication: Synchronous (gRPC/HTTP) vs Asynchronous (RabbitMQ/Kafka).

Javob:

- **Synchronous:** Darhol natija kerak bo'lganda (lekin servislar bir-biriga qat'iy bog'lanib qoladi).
- **Asynchronous:** Event-Driven, servislar mustaqil ishlaydi, yuqori chidamlilik (Resilience) beradi.

76. Circuit Breaker Pattern (Polly library) qanday ishlaydi?

Javob:

Tashqi servis ishlamay qolganda (xatolar soni limitdan oshganda) Circuit Breaker zanjirni uzadi (**Open state**) va so me'rov yubormay darhol xato qaytaradi. Ma'lum vaqtdan so'ng **Half-Open** holatga o'tib sinov so'rovini yuboradi va muvaffaqiyatli bo'lsa **Closed** holatga qaytadi.

77. Saga Pattern: Orchestration vs Choreography distributed transactions uchun.

Javob:

- **Choreography:** Har bir microservice o'z ishini bajarib event uzatadi va boshqa servis event'ni ushlab davom ettiradi (Central controller yo'q).
- **Orchestration:** Markaziy Orchestrator barcha qadamlarni va kompensatsiya (rollback) amallarini boshqaradi.

78. API Gateway Pattern: Ocelot, YARP (Yet Another Reverse Proxy) va Envoy.

Javob:

Mikroservislar oldida yagona kirish nuqtasi bo'lib, Authentication, SSL Termination, Request Aggregation, Rate Limiting va Routing vazifalarini bajaradi. YARP .NET 8 uchun yuqori unumdorlikka ega rasmiy reverse proxy hisoblanadi.

79. SOLID Prinsiplari va C# dagi amaliy ko'rinishlari.

Javob:

- **S:** Single Responsibility (Bitta sinf - bitta mas'uliyat).
- **O:** Open/Closed (Kengaytirish uchun ochiq, o'zgartirish uchun yopiq).
- **L:** Liskov Substitution (Tashqi kod voris sinfni asosiy sinf o'rnida ishlatganda buzilib ketmasligi kerak).
- **I:** Interface Segregation (Katta interfeyslarni kichik va ixtisoslashgan interfeyslarga bo'lish).
- **D:** Dependency Inversion (Yuqori qatlam quyi qatlamga emas, abstraksiyaga bog'lanishi kerak).

80. DRY, KISS, YAGNI va Boy Scout Rule.

Javob:

- **DRY:** Don't Repeat Yourself (Kodni takrorlama).
- **KISS:** Keep It Simple, Stupid (Iloji boricha oddiy qil).
- **YAGNI:** You Ain't Gonna Need It (Hozir kerak bo'lmagan funktsionallikni yozma).
- **Boy Scout Rule:** Kodni har safar o'zingizdan oldingi holatidan ko me'ra tozarroq qoldiring.

81. Repository va Unit of Work Pattern tartibi: EF Core mavjud bo'lganda ular shartmi?

Javob:

EF Core'ning `DbContext` o'zi tayyor **Unit of Work**, `DbSet` esa **Repository** hisoblanadi. Shuning uchun EF Core ustiga qo'shimcha abstrakt Repository qatlami yozish ko'p hollarda **Anti-pattern** (Leaky Abstraction) ga aylanadi. U faqat murakkab testlash yoki orqa fondagi ORM ni almashtirish zarur bo'lganda ishlatiladi.

82. Strategy Pattern va C# DI bilan dinamik tanlash (`IEnumerable`).

Javob:

Bir nechta algoritmlarni (masalan, to me'lov tizimlari: Payme, Click, Uzum) bitta interfeys ortiga yashirish. DI container `IEnumerable` orqali barcha registratsiya qilingan implatsiyalarni taqdim etadi va runtime da keraklisi tanlanadi.

83. Decorator Pattern va Scrutor kutubxonasi orqali Caching/Logging qo'shish.

Javob:

Asosiy servis kodini o'zgartirmasdan uning atrofida qo'shimcha mantiq (keshlash, loglash) o me'rash. `Scrutor` kutubxonasi DI ga `.Decorate()` orqali toza ulash imkonini beradi.

84. Mediator Pattern pros & cons.

Javob:

- **Pros:** Controller va biznes mantiqni to me'liq bo me'ladi, Single Responsibility va Open/Closed prinsiplarini ta'minlaydi.
- **Cons:** Kod bo'laklanadi, so me'rov kim tomonidan bajarilishini topish (tracing) qiyinlashadi.

85. Feature Flags va LaunchDarkly / Microsoft.FeatureManagement.

Javob:

Yangi imkoniyatlarni kodni qayta deploy qilmasdan turib runtime da yoqish/o'chirish imkonini beradi. A/B testing va Blue-Green deployment uchun juda qo'l keladi.

6. Performance Tuning, Diagnostics va Xavfsizlik (86-100)

86. .NET Diagnostics vositalari: `dotnet-trace`, `dotnet-dump`, `dotnet-gcdump`, `dotnet-counters`.

Javob:

- `dotnet-trace` : CPU va metod bajarilish vaqtlarini profayling qilish.
- `dotnet-dump` : Xotiradagi muammoni aniqlash uchun process dump olib tahlil qilish.
- `dotnet-gcdump` : Heap dagi obyektlar va GC xotirasini tahlil qilish.
- `dotnet-counters` : Real vaqt rejimida CPU, RAM, GC, ThreadPool ko'rsatkichlarini kuzatish.

87. BenchmarkDotNet kutubxonasi orqali Microbenchmarking o'tkazish qoidalar.

Javob:

- Release rejimida ishga tushirish.
- `[MemoryDiagnoser]` qo me me'sh orqali Heap allocation'ni o'lchash.
- JIT warm-up va bir nechta iteratsiyalar orqali aniq ko'rsatkich olish.

88. Memory Leak (Xotira sizishi) C# da qanday yuzaga keladi va ularni qanday topish mumkin?

Javob:

C# da manbalar:

- Ishlatilmay qolgan hodisalarga (Events) subscribe bo'lib qolish.
- Static collection'larda obyektlarni ushlab turish.
- `IDisposable` resurslarni yopmaslik.

Topish: `dotnet-gcdump` yoki `dotMemory` yordamida ikkita snapshot olib, farqini (diff) ko'rish.

89. ThreadPool Starvation nima va u qanday alomatlar beradi?

Javob:

ThreadPool da barcha thread'lar sinxron to'siqlar (masalan `.Result`, `.Wait()`) bilan band bo'lganda, yangi asinxron so'rovlar kutish navbatida turib qoladi. Alomatlar: Latency keskin oshib ketadi, CPU past, lekin so'rovlar bajarilmaydi.

90. ArrayPool va MemoryPool orqali GC allocation'ni kamaytirish.

Javob:

Katta hajmdagi bufer array'larni har safar `new byte[1024]` deb yaratmasdan, `ArrayPool.Shared.Rent(1024)` orqali vaqtincha olib, ishlatib bo'lgach `Return()` qilish orqali GC ga tushadigan og'irlik o'rniga xotirani qayta ishlatish.

91. Utf8JsonReader / Utf8JsonWriter va Zero-Allocation JSON Parsing.

Javob:

`System.Text.Json` ning quyi darajadagi strukturasi bo'lgan JSON string'larini `string` ga o'girmasdan to'g'ridan-to'g'ri UTF-8 baytlar (`ReadOnlySpan`) ko'rinishida o'qib olinadi va yuqori tezlik beradi.

92. OWASP Top 10 API Security xatolari va .NET dagi yechimlar.

Javob:

- Broken Object Level Authorization (BOLA) -> Id-based autorizatsiyani har bir so'rovda tekshirish.
- Injection -> Parameterized query (EF Core o'zi himoyalaydi).
- Sensitive Data Exposure -> Encryption, Data Protection API.

93. Data Protection API (IDataProtector) .NET da qanday ishlatiladi?

Javob:

Kritik ma'lumotlarni (Cookie, Token, Kesh parametrlari) shifrlash va desifrlash uchun mo'ljallangan ichki karkas. Kalitlarni avtomatik rotatsiya qiladi.

94. Cross-Site Scripting (XSS) va SQL Injection'dan himoyalash.

Javob:

- XSS: Kiruvchi ma'lumotlarni HTML Encode qilish va `Content-Security-Policy` header'ini yoqish.
- SQL Injection: Raw SQL da parametrli so'rovlar yozish (`FromSqlInterpolated`).

95. Secret Management: User Secrets, Azure Key Vault va HashiCorp Vault.

Javob:

Kod va `appsettings.json` ichida parollar va ulanish kalitlarini saqlamaslik. Development jarayonida `dotnet user-secrets`, Production da esa Environment Variables yoki Secret Vault'lardan foydalanish.

96. Cryptography: Hashing (Argon2, BCrypt, PBKDF2) va Encryption (AES-GCM, RSA) farqi.

Javob:

- **Hashing:** Bir tomonlama operatsiya (Parollarni saqlash uchun Salt bilan ishlatiladi).
- **Encryption:** Ikki tomonlama shifrlash (AES — simmetrik, RSA — asimmetrik).

97. Secure HTTP Headers (HSTS , X-Content-Type-Options , X-Frame-Options , CSP).

Javob:

Middleware orqali brauzerga xavfsizlik buyruqlarini berish:

- **HSTS** : HTTPS ni majburiy qilish.
- **X-Frame-Options: DENY** : Clickjacking'dan himoyalash.
- **CSP** : Ruxsat berilgan skript manbalarini belgilash.

98. Zero-Trust Architecture va Microservice Security.

Javob:

"Ech kachon ishonma, har doim tekshir" tamoyili. Microservice'lar o'zaro muloqot qilganda mTLS (Mutual TLS) va servislar o'rtasida JWT tokenlarni qayta tekshirish.

99. .NET Code Analysis, Roslyn Analyzers va SAST (SonarQube) integratsiyasi.

Javob:

Kompilyatsiya vaqtida kod sifatini, potensial xatoliklarni va xavfsizlik teshiklarini avtomatik tahlil qiluvchi statik analizatorlarni (`EnableNETAnalyzers`) yoqish va CI/CD ga SonarQube bosqichini qo'shish.

100. Senior Developer sifatida Technical Debt (Texnik qarz) va Code Review madaniyatini boshqarish.

Javob:

- Technical Debt'ni refactoring sprintlarida rejalashtirish.
- Code Review'da faqat kod uslubiga emas, arxitektura, performance, testlanuvchanlik va xavfsizlikka e'tibor qaratish.
- Automated CI/CD (Linters, Unit Tests, Integration Tests) orqali inson omilini kamaytirish.

Hujjat Senior .NET Developerlar uchun maxsus tayyorlandi.